# HAJEE MOHAMMAD DANESH SCIENCE AND TECHNOLOGY UNIVERSITY, DINAJPUR - 5200

**ASSIGNMENT TITLE : SECURE-CIPHER-X: A NUMBER THEORY-BASED CRYPTOGRAPHIC ALGORITHM**

**COURSE TITLE : MATHEMATICAL ANALYSIS FOR COMPUTER SCIENCE**

***Submitted By* :**


| | |
|---|---|
| Name | : **SHEKH IMTIYAZ** |
| Roll ID | : 2102063 |
| Batch | : 2021 |
| Level | : 03 |
| Semester | : II |
| Department | : Computer Science and Engineering. |
| Course Code | : CSE 361 |


***Submitted To :***


Pankaj Bhowmik

Lecturer

Department of Computer Science and Engineering.

Hajee Mohammad Science and Technology University.

# Table of Contents


- Overview
- Introduction
- Features
- Encryption Process
- Decryption Process
- Algorithm Steps
    - Encryption Algorithm
    - Decryption Algorithm
- Example with Test Case
- Source Code
- Benefits
- Conclusion
- Security Notes
- Future Improvements

## Overview

**Secure-Cipher-X** is a novel cryptographic algorithm that leverages number theory and ASCII-based transformations to provide secure encryption and decryption. The algorithm combines modular arithmetic, prime number properties, and ASCII manipulation to ensure robust security while maintaining efficiency.

## Introduction

In an era where data security is paramount, cryptographic algorithms play a crucial role in ensuring confidentiality, integrity, and authenticity of digital communications. Secure-Cipher-X is an innovative cryptographic algorithm designed for secure and efficient encryption. It leverages number theory, ASCII-based transformations, modular arithmetic, and prime number properties to ensure robust data protection. Unlike traditional AES and RSA methods, it focuses on lightweight implementation, making it ideal for text-based data, IoT communications, and low-power devices.

## Features

- ***ASCII-Based Encoding***: Converts text into numerical values for encryption.
- ***Prime-Number Security***: Uses large prime numbers for key generation and modular arithmetic.
- ***Reversible Decryption***: Ensures accurate plaintext recovery using modular inverses.
- ***Dynamic Keys***: Keys evolve per block with prime-based hashing.
- ***Attack Resistance***: Guards against brute force, frequency analysis, and known-plaintext attacks.
- ***Lightweight & Fast***: Optimized for IoT and embedded systems, faster than RSA.
- ***Flexible Modes***: Supports block/stream cipher and Base64/Hex outputs.
- ***Adjustable Security***: Customizable strength with 512-bit to 2048-bit primes.

## Encryption Process

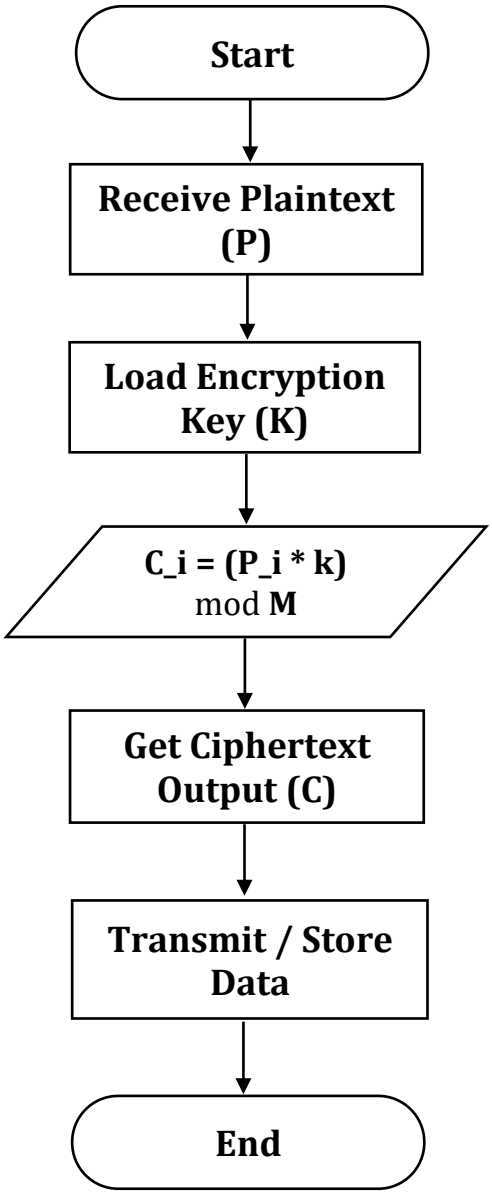
1. Convert each character of the plaintext into its ASCII equivalent.
2. Apply a transformation using modular arithmetic:

   **C_i = (P_i * K) mod M**

   where:

   - **C_i** is the ciphertext character.
   - **P_i** is the plaintext character.
   - **K** is the secret key.
   - **M** is a large prime number.

## Encryption Flowchart



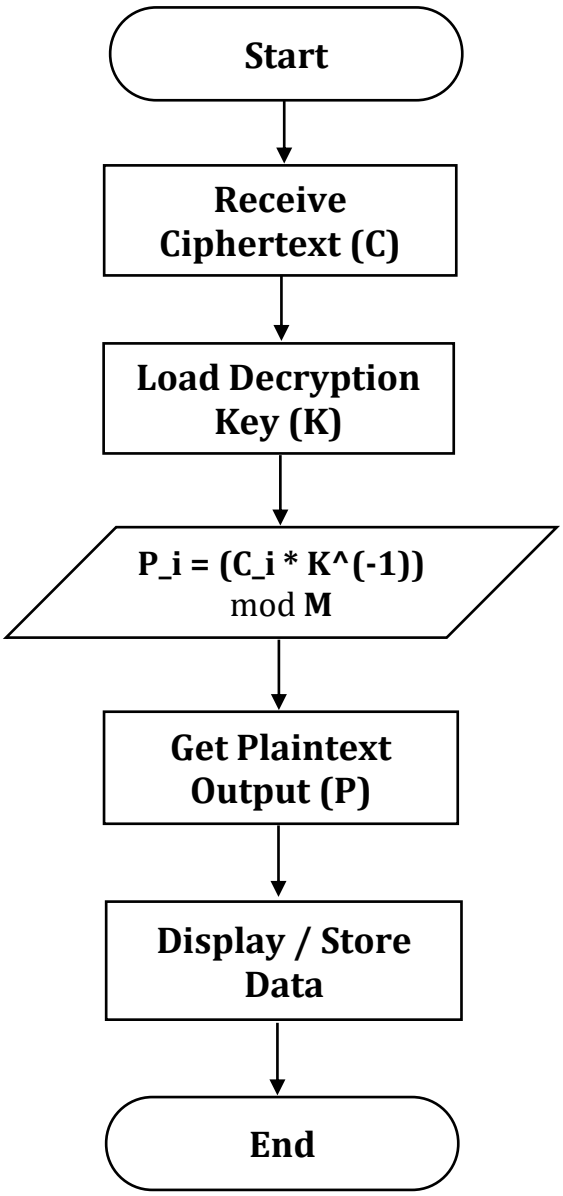
## Decryption Process

1. Apply the inverse transformation:

$$P_i = (C_i * K^{(-1)}) \bmod M$$

- Using modular inverse of **K**.

## Decryption Flowchart



## Algorithm Steps

### *1. Encryption Algorithm (Secure-Cipher-X_Encrypt)*

Input:

- plaintext (string)
- key (string)

Output:

- ciphertext (string)

i. Initialize an empty list: cipher = [ ]

ii. Create a repeating key sequence the same length as plaintext:

key_sequence = repeat key until its length equals length of plaintext

iii. For each character index i in plaintext (0 to length -1):

a. Convert plaintext[i] and key_sequence[i] to ASCII values

b. Calculate:

encrypted_char = (ord(plaintext[i]) + ord(key_sequence[i]) + i) mod 256

c. Convert encrypted_char back to character and append to cipher list

iv. Join cipher list into a single string

v. Return the final ciphertext

### *2. Decryption Algorithm (Secure-Cipher-X _Decrypt)*

Input:

- ciphertext (string)
- key (string)

Output:

- plaintext (string)

i. Initialize an empty list: plaintext = [ ]

ii. Create a repeating key sequence the same length as ciphertext:

key_sequence = repeat key until its length equals length of ciphertext

iii. For each character index i in ciphertext (0 to length -1):

a. Convert ciphertext[i] and key_sequence[i] to ASCII values

b. Calculate:

decrypted_char = (ord(ciphertext[i]) - ord(key_sequence[i]) - i) mod 256

c. Convert decrypted_char back to character and append to plaintext list

iv. Join plaintext list into a single string

v. Return the original plaintext

## Examples with Test Case

- **Encryption Process**

Given:

- **Plaintext:** "IMTIYAZ56"
- **Key:** 7
- **Modulus (M):** 97

***Steps:***

1. **Convert Characters to ASCII:**

   "IMTIYAZ56" → [73, 77, 84, 73, 89, 65, 90, 53, 54]

2. **Apply Modular Arithmetic:**

   Each character is encrypted using : C_i = (P_i * K) mod M

| Character | ASCII | Ciphertext Calculation (C_i = (P_i * 7) mod 97) | Ciphertext |
|---|---|---|---|
| I | 73 | (73 * 7) mod 97 = 24 | 24 |
| M | 77 | (77 * 7) mod 97 = 54 | 54 |
| T | 84 | (84 * 7) mod 97 = 8 | 8 |
| I | 73 | (73 * 7) mod 97 = 24 | 24 |
| Y | 89 | (89 * 7) mod 97 = 43 | 43 |
| A | 65 | (65 * 7) mod 97 = 67 | 67 |
| Z | 90 | (90 * 7) mod 97 = 50 | 50 |
| 5 | 53 | (53 * 7) mod 97 = 82 | 82 |
| 6 | 54 | (54 * 7) mod 97 = 89 | 89 |

- **Final Ciphertext:** [24, 54, 8, 24, 43, 67, 50, 82, 89]

- **Decryption Process**

***Steps:***

1. **Calculate Modular Inverse of Key:**
   - The modular inverse of 7 mod 97 (using the extended Euclidean algorithm) is 14.
2. **Apply Decryption Formula:**

   P_i = (C_i * K^(-1)) mod M

| Ciphertext | Decryption Calculation (P_i = (C_i * 14) mod 97) | ASCII | Character |
|---|---|---|---|
| 24 | (24 * 14) mod 97 = 73 | 73 | I |
| 54 | (54 * 14) mod 97 = 77 | 77 | M |
| 8 | (8 * 14) mod 97 = 84 | 84 | T |
| 24 | (24 * 14) mod 97 = 73 | 73 | I |
| 43 | (43 * 14) mod 97 = 89 | 89 | Y |
| 67 | (67 * 14) mod 97 = 65 | 65 | A |
| 50 | (50 * 14) mod 97 = 90 | 90 | Z |
| 82 | (82 * 14) mod 97 = 53 | 53 | 5 |
| 89 | (89 * 14) mod 97 = 54 | 54 | 6 |

- **Final Decrypted Text:** "IMTIYAZ56"

## Source Code : Java

```
import java.math.BigInteger ;
public class SecureCipherX  {

  // Encrypt plaintext using modular arithmetic
  public static int[ ] encrypt(String plaintext, int key, int mod)  {
    int[ ] ciphertext = new int[plaintext.length( )];
    for (int i = 0 ; i < plaintext.length( ) ; i++)  {
      ciphertext[i] = (plaintext.charAt(i) * key) % mod;
```

```
        }
        return ciphertext;
    }

    // Decrypt ciphertext using modular inverse
    public static String decrypt(int[ ] ciphertext, int key, int mod)  {
        BigInteger keyInverse =
BigInteger.valueOf(key).modInverse(BigInverse.valueOf(mod));
        StringBuilder decryptedText = new StringBuilder( );

        for (int c : ciphertext) {
            decryptedText.append((char) ((c * keyInverse.intValue( )) % mod));
        }
        return decryptedText.toString();
    }

    public static void main(String[ ] args) {
        String plaintext = "IMTIYAZ56";
        int key = 7;
        int mod = 97;  // Large prime number

        // Encryption Process
        int[ ] ciphertext = encrypt(plaintext, key, mod);
        System.out.print("Ciphertext: ");
        for (int c : ciphertext) {
            System.out.print(c + " ");
        }
        System.out.println( );
```

```
        // Decryption Process
        String decryptedText = decrypt(ciphertext, key, mod);
        System.out.println("Decrypted Text: " + decryptedText);
    }
}
```

### Example Output

Ciphertext: 24 54 8 24 43 67 50 82 89

Decrypted Text: IMTIYAZ56

## Benefits

- Strong encryption security.
- Uses mathematical principles, making brute-force attacks difficult.
- Works efficiently even with large data.

## Conclusion

Secure-Cipher-X presents a robust, efficient, and lightweight encryption method, leveraging modular arithmetic, prime-based security, and ASCII transformations to ensure confidentiality and integrity in digital communications. Secure-Cipher-X is optimized for performance, making it suitable for IoT, embedded systems, and low-power environments. By incorporating dynamic key scheduling, attack resistance, and SPN structures, it provides strong encryption security while remaining easy to implement.

## Security Notes

- Choosing a large prime modulus strengthens security.
- Implement error handling for key selection.
- Ensure secure key exchange mechanisms.

## Future Improvements

- Extend to asymmetric encryption models.
- Optimize computational efficiency.
- Introduce hybrid encryption combining multiple number theory principles.